

PRACTICAL WEEK 8 - SESSION 15

Task 1 :- To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
GNU nano 7.2 task1_combined_redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int file;
    int saved_stdout;
    int saved_stderr;

    printf("Combined Redirection Demonstration\n");

    file = open("combined_output.txt",
                O_WRONLY | O_CREAT | O_TRUNC,
                0644);

    if (file == -1) {
        perror("combined_output.txt");
        return 1;
    }

    saved_stdout = dup(STDOUT_FILENO);
    saved_stderr = dup(STDERR_FILENO);

    dup2(file, STDOUT_FILENO);
    dup2(STDOUT_FILENO, STDERR_FILENO);

    close(file);

    printf("Message 1: Standard output\n");
    fprintf(stderr, "Message 2: Standard error\n");
    printf("Message 3: Output after redirection\n");
    fprintf(stderr, "Message 4: Error after redirection\n");

    fflush(stdout);
    fflush(stderr);

    dup2(saved_stdout, STDOUT_FILENO);
    dup2(saved_stderr, STDERR_FILENO);

    close(saved_stdout);
    close(saved_stderr);

    printf("Streams restored successfully.\n");

    return 0;
}
```

```

root@HP:~/skill_week8# nano task1_combined_redirect.c
root@HP:~/skill_week8# gcc task1_combined_redirect.c -o task1_combined_redirect
root@HP:~/skill_week8# ./task1_combined_redirect
Combined Redirection Demonstration
Streams restored successfully.
root@HP:~/skill_week8# cat combined_output.txt
Message 1: Standard output
Message 2: Standard error
Message 3: Output after redirection
Message 4: Error after redirection
root@HP:~/skill_week8# |

```

Task 2 :- To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    int file;
    pid_t p1, p2;

    printf("Complex Command Execution\n");
    printf("Pipeline: printf | tr > result.txt\n");

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("printf", "printf",
              "banana\napple\norange\n",
              (char *)NULL);

        perror("printf");
        exit(1);
    }

    p2 = fork();

```

Scenarios

```

    if (p2 == 0) {
        close(pipefd[1]);

        file = open("result.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

        if (file == -1) {
            perror("result.txt");
            exit(1);
        }

        dup2(pipefd[0], STDIN_FILENO);
        dup2(file, STDOUT_FILENO);

        close(pipefd[0]);
        close(file);

        execlp("tr", "tr", "a-z", "A-Z",
               (char *)NULL);

        perror("tr");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}

```

```

root@HP:~/skill_week8# gcc task2_complex_pipeline.c -o task2_complex_pipeline
root@HP:~/skill_week8# ./task2_complex_pipeline
Complex Command Execution
Pipeline: printf | tr > result.txt
Pipeline execution completed.
root@HP:~/skill_week8# cat result.txt
BANANA
APPLE
ORANGE
root@HP:~/skill_week8# |

```

Observation :- Combined redirection, pipes, file-descriptor duplication, and complex command execution were implemented and tested successfully.

SESSION 16

Task 1 :- To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
GNU nano 7.2 task3_background_jobs.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Background Job Demonstration\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background job started. PID = %d\n", getpid());

        sleep(3);

        printf("Background job completed. PID = %d\n", getpid());

        exit(0);
    }

    printf("Job stored with PID = %d\n", pid);
    printf("Parent continues without blocking.\n");

    sleep(1);

    printf("Checking background job status...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("Job completed normally.\n");

    return 0;
}
```

```
root@HP:~/skill_week8# gcc task3_background_jobs.c -o task3_background_jobs
root@HP:~/skill_week8# ./task3_background_jobs
Background Job Demonstration
Job stored with PID = 545
Parent continues without blocking.
Background job started. PID = 545
Checking background job status...
Background job completed. PID = 545
Job completed normally.
root@HP:~/skill_week8# |
```

Task 2 :- To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
GNU nano 7.2 task4_jo
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 3

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

int main() {
    struct Job jobs[MAX_JOBS];
    int count = 0;
    int status;
    pid_t pid;

    printf("Job Table Demonstration\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(2);
        exit(0);
    }

    jobs[count].id = count + 1;
    jobs[count].pid = pid;
    snprintf(jobs[count].state,
             sizeof(jobs[count].state),
             "RUNNING");
    count++;

    printf("\nJob Table:\n");
    printf("ID\tPID\tSTATE\n");
```

```

printf("ID\tPID\tSTATE\n");

for (int i = 0; i < count; i++) {
    printf("%d\t%d\t%s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

waitpid(pid, &status, 0);

snprintf(jobs[0].state,
         sizeof(jobs[0].state),
         "COMPLETED");

printf("\nUpdated Job Table:\n");
printf("ID\tPID\tSTATE\n");

for (int i = 0; i < count; i++) {
    printf("%d\t%d\t%s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

if (WIFEXITED(status))
    printf("\nJob record validated successfully.\n");

return 0;
}
|
root@HP:~/skill_week8# gcc task4_job_table.c -o task4_job_table
root@HP:~/skill_week8# ./task4_job_table
Job Table Demonstration

Job Table:
ID      PID      STATE
1       556      RUNNING

Updated Job Table:
ID      PID      STATE
1       556      COMPLETED

Job record validated successfully.
root@HP:~/skill_week8# |

```

Observation :- Background job execution and job-table management were implemented successfully, demonstrating process tracking, status updates, and completion handling.